Viaduct

an extensible, optimizing compiler for secure multi-party computation

Rolph Recto

Cornell PL Retreat 2019

Coşku Acay

Andrew Myers

Josh Gancher

Elaine Shi

viaduct

(n.) *a long bridge-like structure, typically a series of arches, carrying a road or railroad across a valley or other low ground.*

an extensible, optimizing compiler for ***secure multi-party computation***

allow mutually distrusting principals to perform computations...

an extensible, optimizing ***compiler*** *for secure multi-party computation*

allow mutually distrusting principals to perform computations...

....written in a high-level specification language...

*an **extensible**, optimizing compiler for secure multi-party computation*

allow mutually distrusting principals to perform computations...

....written in a high-level specification language...

....compiled to protocols using a generic security analysis...

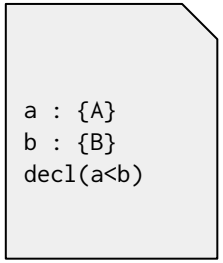
*an extensible, **optimizing** compiler for secure multi-party computation*

allow mutually distrusting principals to perform computations...

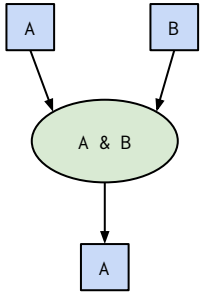
....written in a high-level specification language...

....compiled to protocols using a generic security analysis...

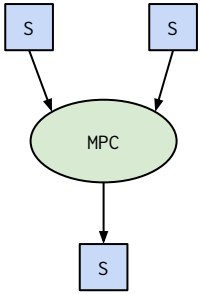
....that uses expensive crypto only when necessary.



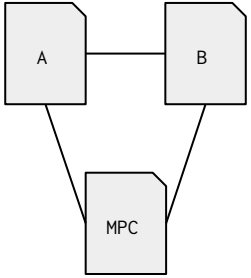
security specification



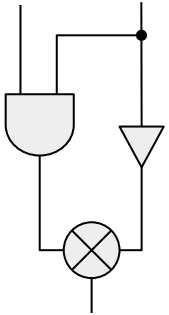
computing security reqs



protocol selection

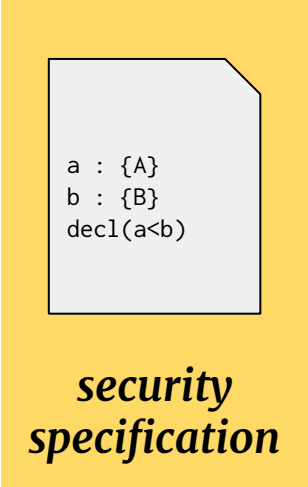


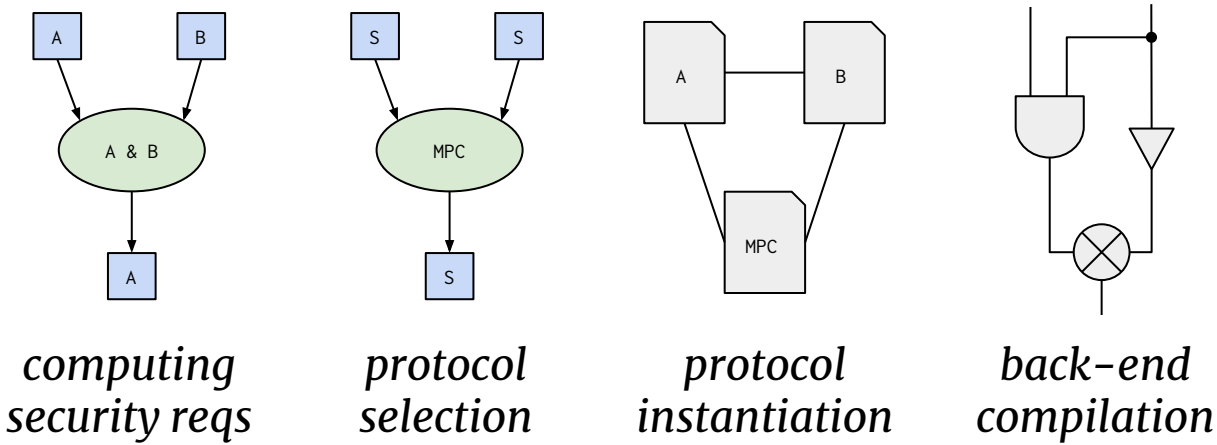
protocol instantiation



back-end compilation

compiler architecture





compiler architecture

specification language

- specifies high-level security policy
- simple imperative language with arrays, security labels, and downgrading
- type system ensures **secure information flow**

“dangerous” millionaire’s game

```
a : int{A & B←}
b : int{B & A←}
b_richer : bool{A⊓B}

b_richer = declassify(a < b, A⊓B)
if (b_richer) {
  a = 0;
} else {
  b = 0;
}
```

Alice and Bob check who is richer without revealing actual net worth. The poorer person loses all her money.

Alice and Bob trust each other, but cannot read the other’s private data

FLAM label model

[Arden et al, CSF 2015]

A→

- secret to A

A←

- trusted by A

A & B

- trusted by and secret to both A and B

A ⊓ B

- trusted by and visible to either A and B

security policies represented by principals and label operators

“dangerous” millionaire’s game

```
a : int{A & B←}
b : int{B & A←}
b_richer : bool{A⊓B}

b_richer = declassify(a < b, A⊓B)
if (b_richer) {
  a = 0;
} else {
  b = 0;
}
```

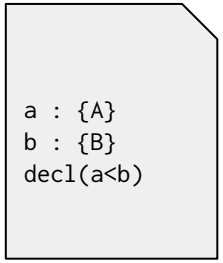
Alice and Bob trusts, only Alice can Read

Alice and Bob trusts, only Bob can Read

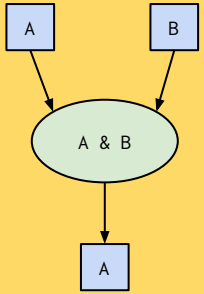
Both Alice and Bob trust and can read

*Need **both** Alice and Bob to read*

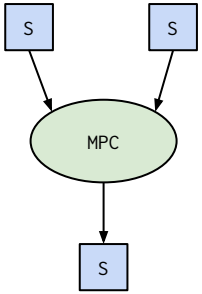
*Allow **either** Alice or Bob to read (make less secret)*



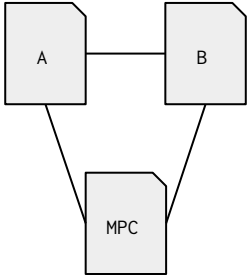
security specification



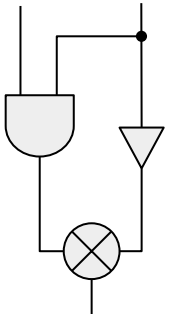
computing security reqs



protocol selection



protocol instantiation



back-end compilation

compiler architecture

computing security requirements

- what are security requirements for protocols to **store variables** and **perform computations**?
- idea: solve information flow constraints from specification program
- best solution comes as **minimum authority needed** for program expressions

computing security requirements

information flow labels

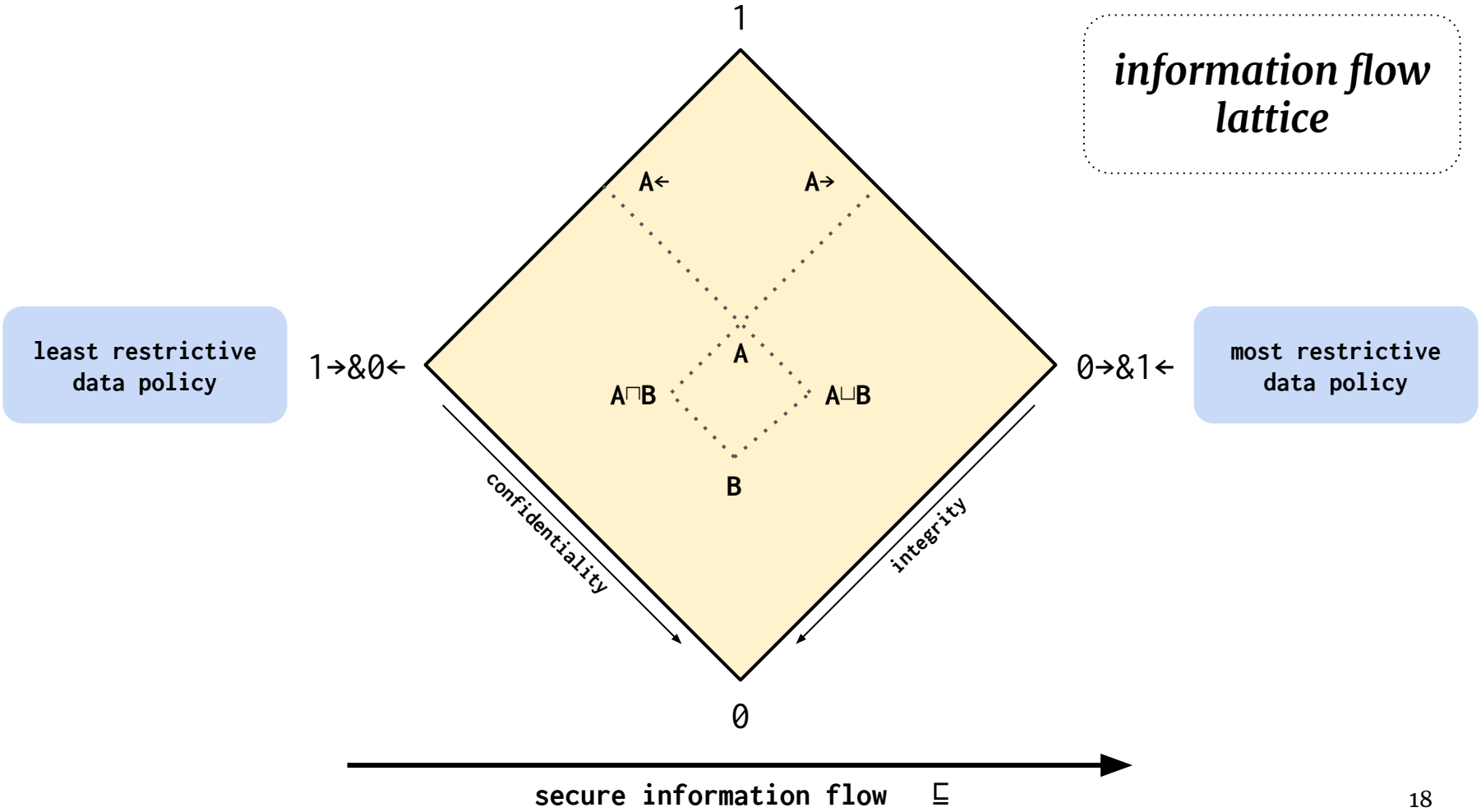
- classifies policies regarding the use of *data*
- describes who can read and influence data
- ordered by restrictiveness of data policy (flows-to)

authority labels

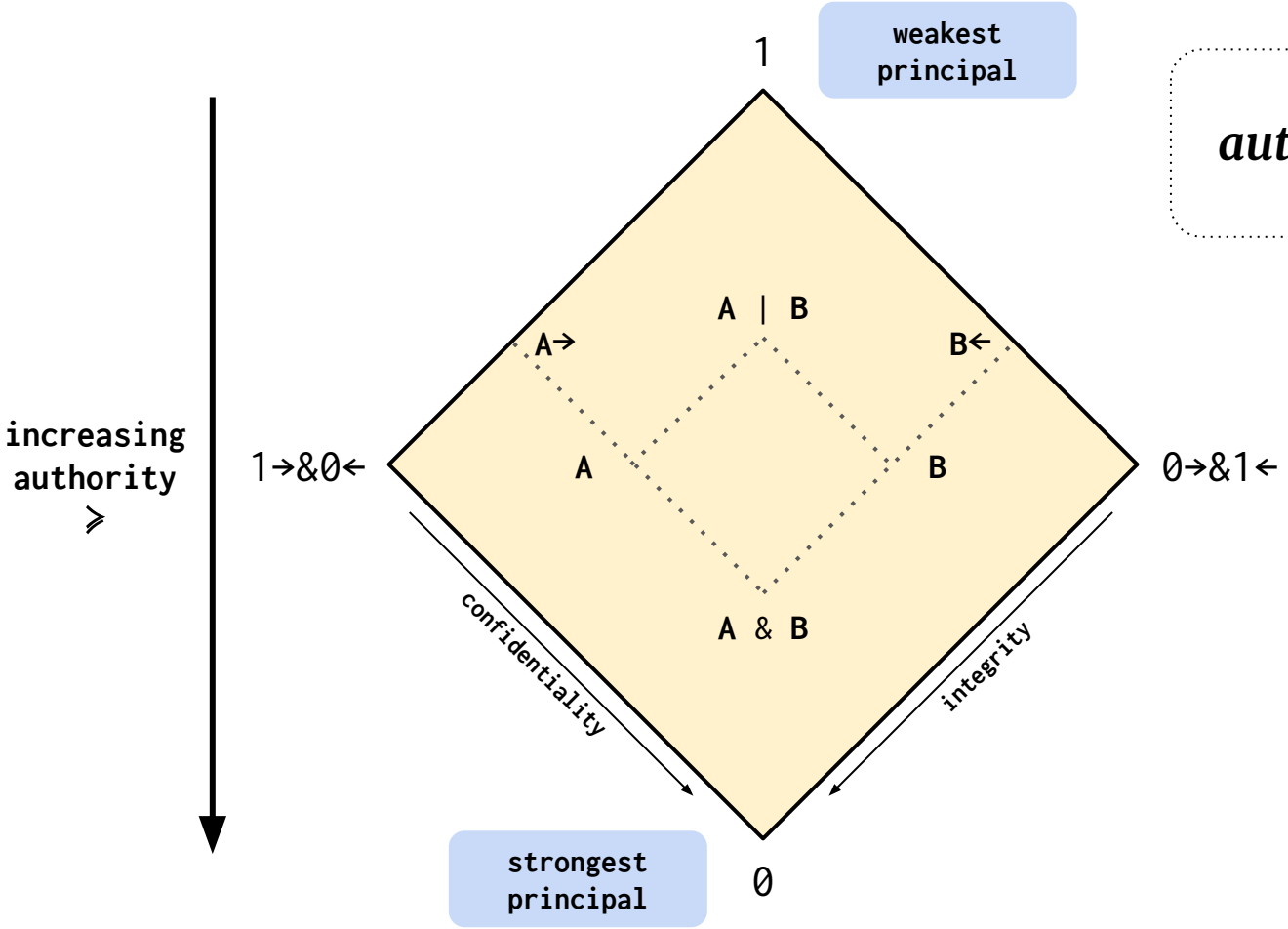
- classifies the power of *principals* in a system
- describes what a principal can read and influence
- ordered by what the amount of principal's authority (acts-for)

can both be represented by FLAM labels

information flow lattice



authority lattice



security label constraints

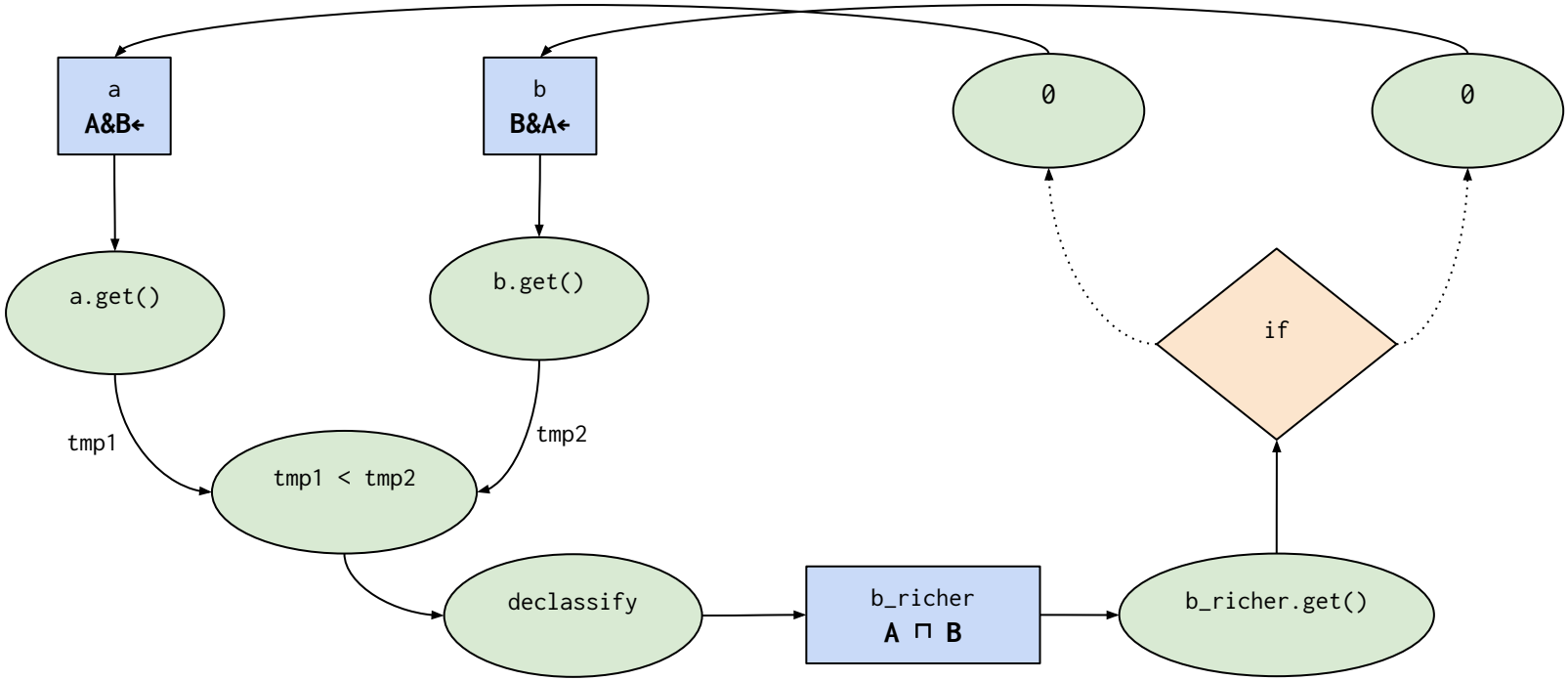
- spec program induces constraints for secure information flow

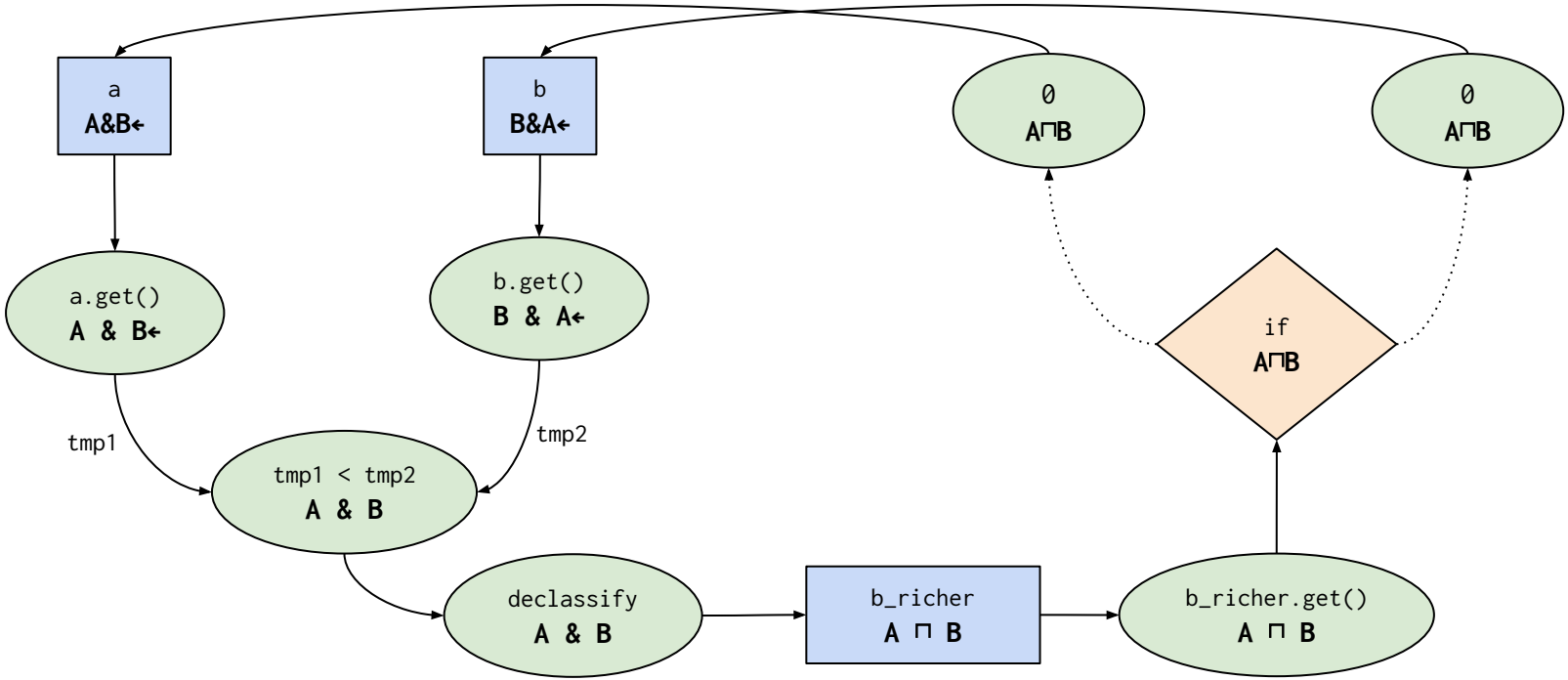
  $x = y \implies L(y) \sqsubseteq L(x)$

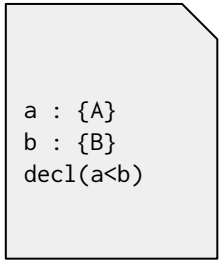
- convert into authority constraints

  $L(y) \sqsubseteq L(x) \implies L(x)\rightarrow \succcurlyeq L(y)\rightarrow,\ L(y)\leftarrow \succcurlyeq L(x)\leftarrow$

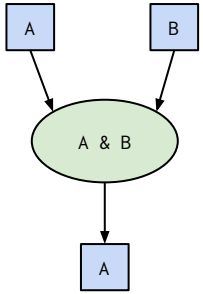
- solve for greatest fixpoint (least authority) solution using iterative worklist algorithm


a
A&B←
b
B&A←
0
0
a.get()
b.get()
if
tmp1
tmp2
tmp1 < tmp2
declassify
b_richer
A ⊓ B
b_richer.get()

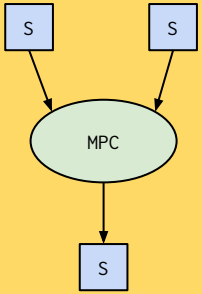

a
A&B←
b
B&A←
0
A⊓B
0
A⊓B
a.get()
A & B←
b.get()
B & A←
if
A⊓B
tmp1
tmp2
tmp1 < tmp2
A & B
declassify
A & B
b_richer
A ⊓ B
b_richer.get()
A ⊓ B



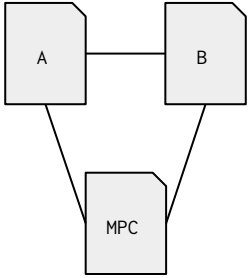
security specification



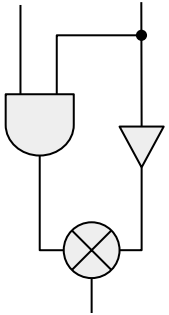
computing security reqs



protocol selection



protocol instantiation



back-end compilation

compiler architecture

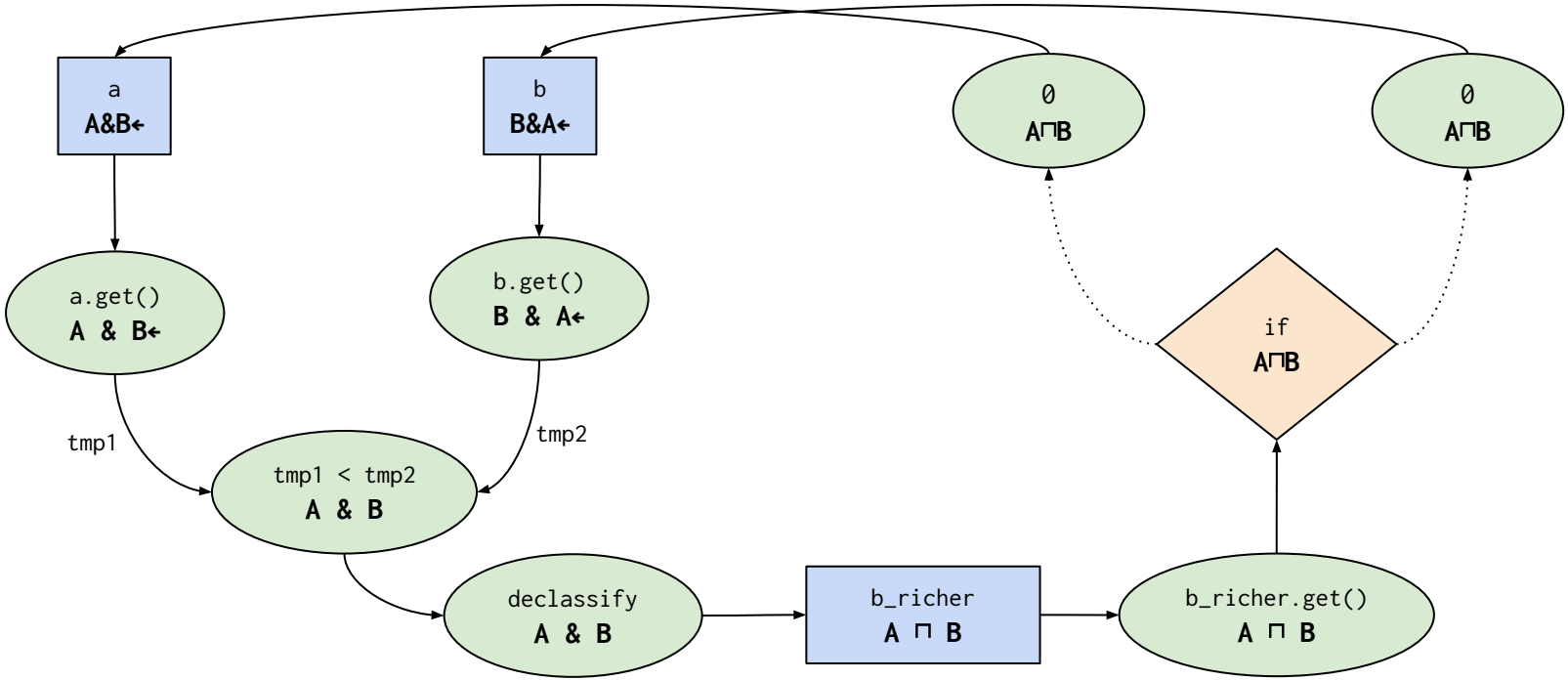
protocol selection

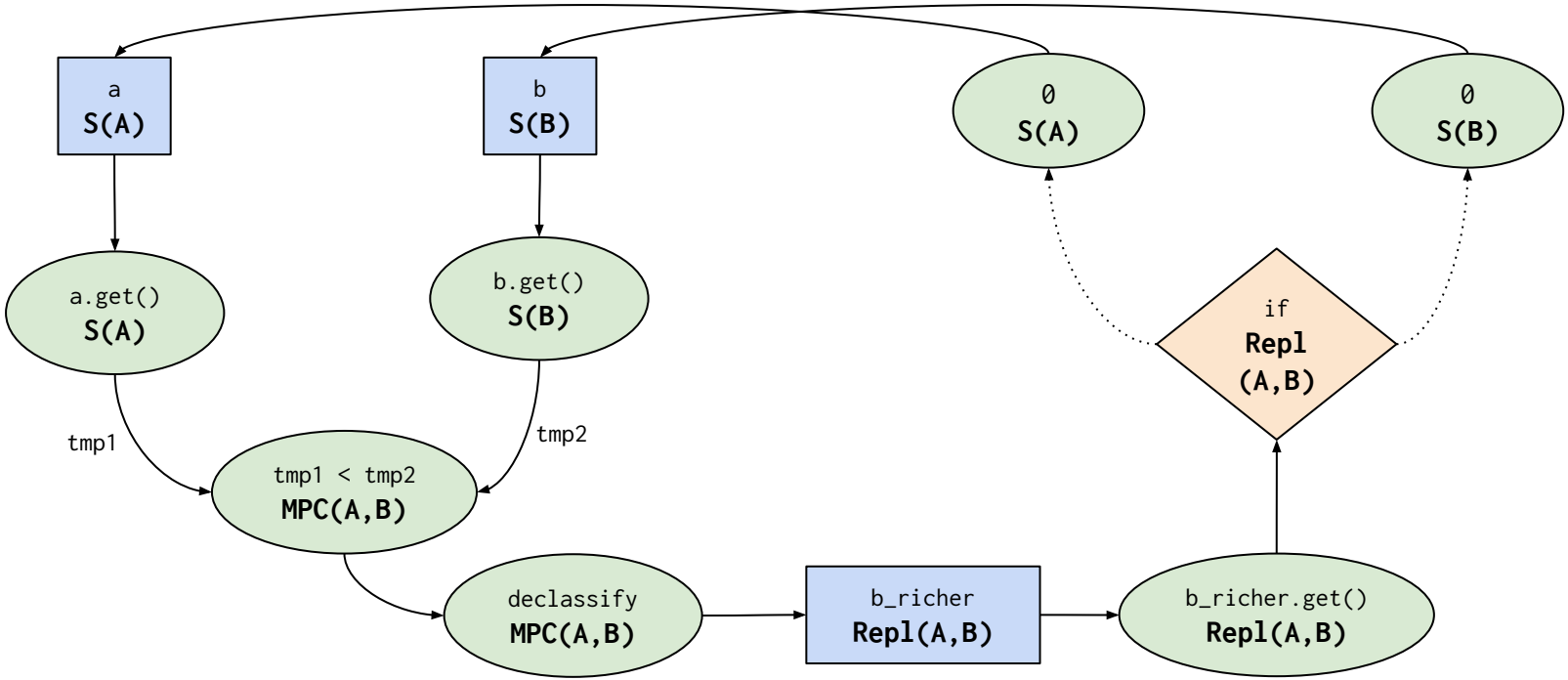
- protocols must be able to “act for” program components
- derive authority of protocol from authority of protocol participants

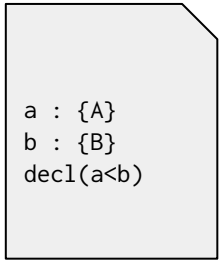
Protocol	Authority	Description
`Single(A)`	`L(A)`	store data or perform computation in cleartext on one host
`Repl(A,B)`	`L(A)` ⊓ `L(B)`	replicate data or computation in cleartext on multiple hosts
`MPC(A,B)`	`L(A) & L(B)`	hosts perform MPC protocol

protocol selection

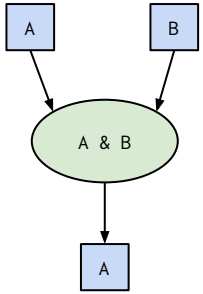
- authority requirements satisfied by many protocols, use cost model to choose optimal
- model cost as required communication
- use directed search (Dijkstra's) to find optimal


a
A&B←
b
B&A←
0
A⊓B
0
A⊓B
a.get()
A & B←
b.get()
B & A←
if
A⊓B
tmp1
tmp2
tmp1 < tmp2
A & B
declassify
A & B
b_richer
A ⊓ B
b_richer.get()
A ⊓ B

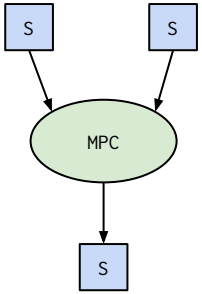

a
S(A)
b
S(B)
0
S(A)
0
S(B)
a.get()
S(A)
b.get()
S(B)
if
Repl
(A,B)
tmp1
tmp2
tmp1 < tmp2
MPC(A,B)
declassify
MPC(A,B)
b_richer
Repl(A,B)
b_richer.get()
Repl(A,B)



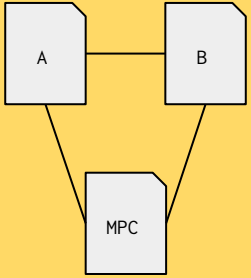
security specification



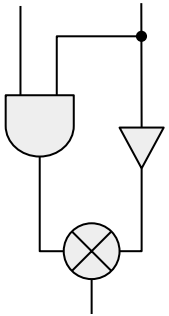
computing security reqs



protocol selection



protocol instantiation



back-end compilation

compiler architecture

protocol instantiation

- compile process configuration from selected protocol map
- process configuration represents a *hybrid protocol* - cryptographic primitives left abstract

```
process alice {
  int a
  bool b_richer
  send a to MPC
  res1 <- recv MPC
  b_richer = res1
  if (b_richer) {
    a = 0
  } else {}
}
```

```
process bob {
  int b
  bool b_richer
  send b to MPC
  res2 <- recv MPC
  b_richer = res2
  if (b_richer) {}
  else {
    b = 0
  }
}
```

```
process MPC {
  ta <- recv alice
  tb <- recv bob
  let res = ta < tb
  send res to alice
  send res to bob
}
```

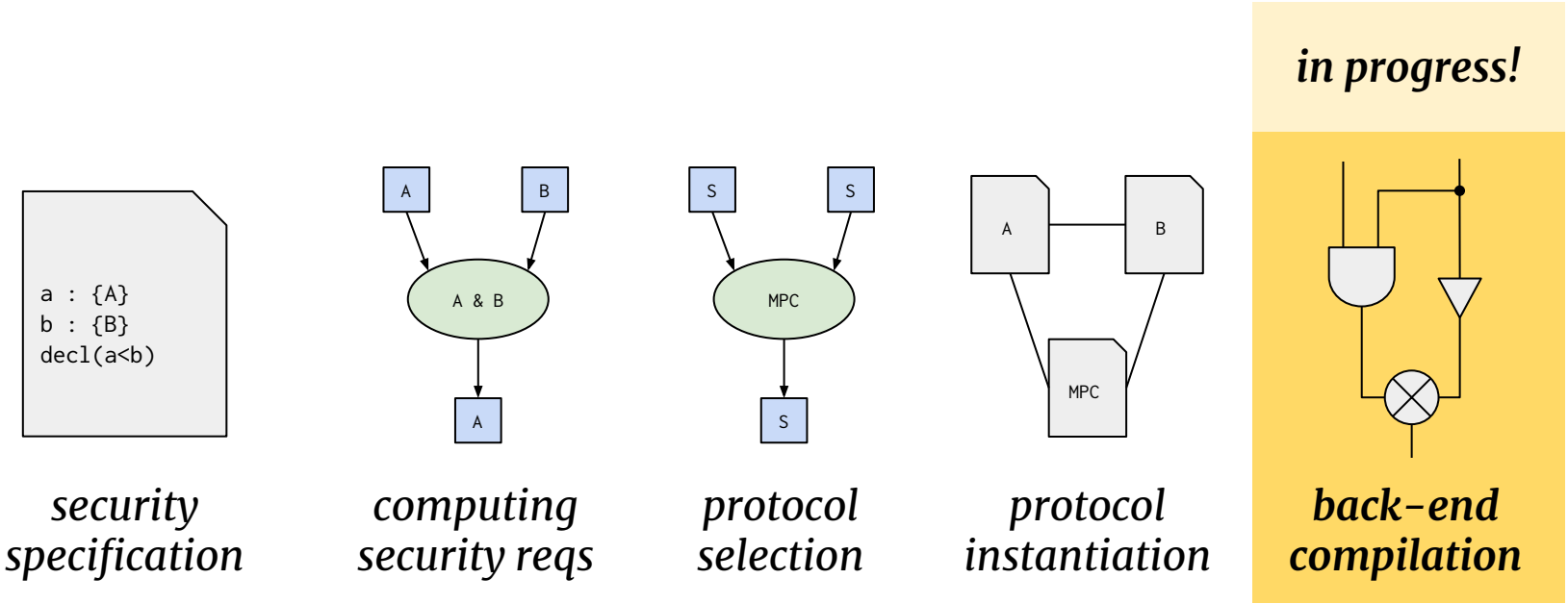
```
process alice {
  int a
  bool b_richer
  send a to MPC
  res1 <- recv MPC
  b_richer = res1
  if (b_richer) {
    a = 0
  } else {}
}
```

```
process bob {
  int b
  bool b_richer
  send b to MPC
  res2 <- recv MPC
  b_richer = res2
  if (b_richer) {}
  else {
    b = 0
  }
}
```

compiled to cleartext

```
process MPC {
  ta <- recv alice
  tb <- recv bob
  let res = ta < tb
  send res to alice
  send res to bob
}
```

compiled to MPC protocol (garbled circuit, etc)



compiler architecture

work in progress

- formal correctness proof for generated protocols
- backend implementation
- more protocols
- richer specification language

thanks!